

SOLID Design Principles

Part 2: LSP Guidelines, ISP, and DIP

1. Liskov Substitution Principle (LSP): The Deep Dive

We previously defined LSP as ensuring that a derived class can be substituted for its base class without breaking the application. Because this is the most frequently violated principle in production systems, industry standards dictate three specific sets of guidelines—**Signature, Property, and Method rules**—to guarantee a subclass is truly substitutable.

A. The Signature Rules

These rules dictate how overriding method signatures (arguments, return types, and exceptions) must behave in a child class.

Rule	Constraint on Child Class	Example / Explanation
Method Argument Rule	Must be the same or broader (Superclass).	If Parent expects a <code>String</code> , Child cannot expect an <code>Integer</code> . <i>(Note: Strongly-typed languages like C++/Java enforce exact matches for overriding, so this is naturally prevented).</i>
Return Type Rule (Covariance)	Must be the same or narrower (Subclass).	If Parent returns an <code>Animal</code> , Child can safely return a <code>Dog</code> . Child cannot return a broader type like <code>Organism</code> , because the client expecting an <code>Animal</code> might crash.
Exception Rule	Can only throw the same or narrower exceptions.	If Parent throws a <code>LogicError</code> , Child can throw an <code>OutOfRangeException</code> (a subclass). Child cannot throw a <code>RuntimeError</code> , because the client's <code>catch(LogicError e)</code> block will miss it, causing a crash.

B. The Property Rules

- **Class Invariants:** An invariant is a truth that must hold for the entire life of the object. For example, `balance >= 0` in a Bank Account. A child class must *respect or strengthen* this invariant; it cannot weaken it (e.g., a `CheatAccount` cannot allow negative balances).
- **History Constraint:** A child class cannot modify immutable properties defined by the parent. If a parent class dictates that "withdrawal is always allowed", a child class (like a `FixedDepositAccount`) cannot suddenly disable withdrawals by throwing an exception.

C. The Method Rules (Pre and Post Conditions)

Think of these as the input/output expectations of the methods.

- **Pre-conditions (Inputs):** Must be the **same or weaker**. If the Parent requires a password to be `>= 8` characters, the Child can weaken it to `>= 6` characters. The client (who expects `>= 8`) will still be satisfied. The Child *cannot* strengthen it to `>= 10`.
- **Post-conditions (Outputs):** Must be the **same or stronger**. If the Parent's `brake()` method guarantees speed is reduced, an `ElectricCar` child must reduce speed AND can optionally strengthen the post-condition by also charging the battery. It *cannot* fail to reduce speed.

2. Interface Segregation Principle (ISP)

Definition: "Many client-specific interfaces are better than one general-purpose interface. Clients should not be forced to implement methods they do not use."

The Anti-Pattern: Fat Interfaces

Suppose we define an interface `Shape` with methods `calculateArea()` and `calculateVolume()`. If a `Square` class implements `Shape`, it is forced to implement `calculateVolume()`. Since a 2D square has no volume, the developer has to leave the method blank or throw a "Not Applicable" exception. This creates dirty, misleading code.

The Solution: Segregate the Interfaces

Break the "fat" interface into smaller, logically cohesive interfaces.

```
// Segregated Interfaces
class TwoDimensionalShape {
public:
    virtual double calculateArea() = 0;
};

class ThreeDimensionalShape : public TwoDimensionalShape {
public:
    virtual double calculateVolume() = 0;
};

// Clean Implementations
class Square : public TwoDimensionalShape {
    double calculateArea() override { return side * side; }
    // No volume method forced here!
};

class Cube : public ThreeDimensionalShape {
    double calculateArea() override { return 6 * side * side; }
    double calculateVolume() override { return side * side * side; }
};
```

3. Dependency Inversion Principle (DIP)

Definition: "High-level modules should not depend on low-level modules. Both should depend on abstractions (interfaces)."

High-level modules encapsulate the core business logic (e.g., an Application Service). Low-level modules execute basic tasks (e.g., Disk I/O, Database SQL Queries).

The Problem: Tight Coupling

If your UserService (High-Level) directly instantiates and calls methods on MySQLDatabase (Low-Level), your business logic is chained to MySQL. If the company migrates to MongoDB or Cassandra, you must rewrite the business logic. This inherently breaks the Open/Closed Principle!

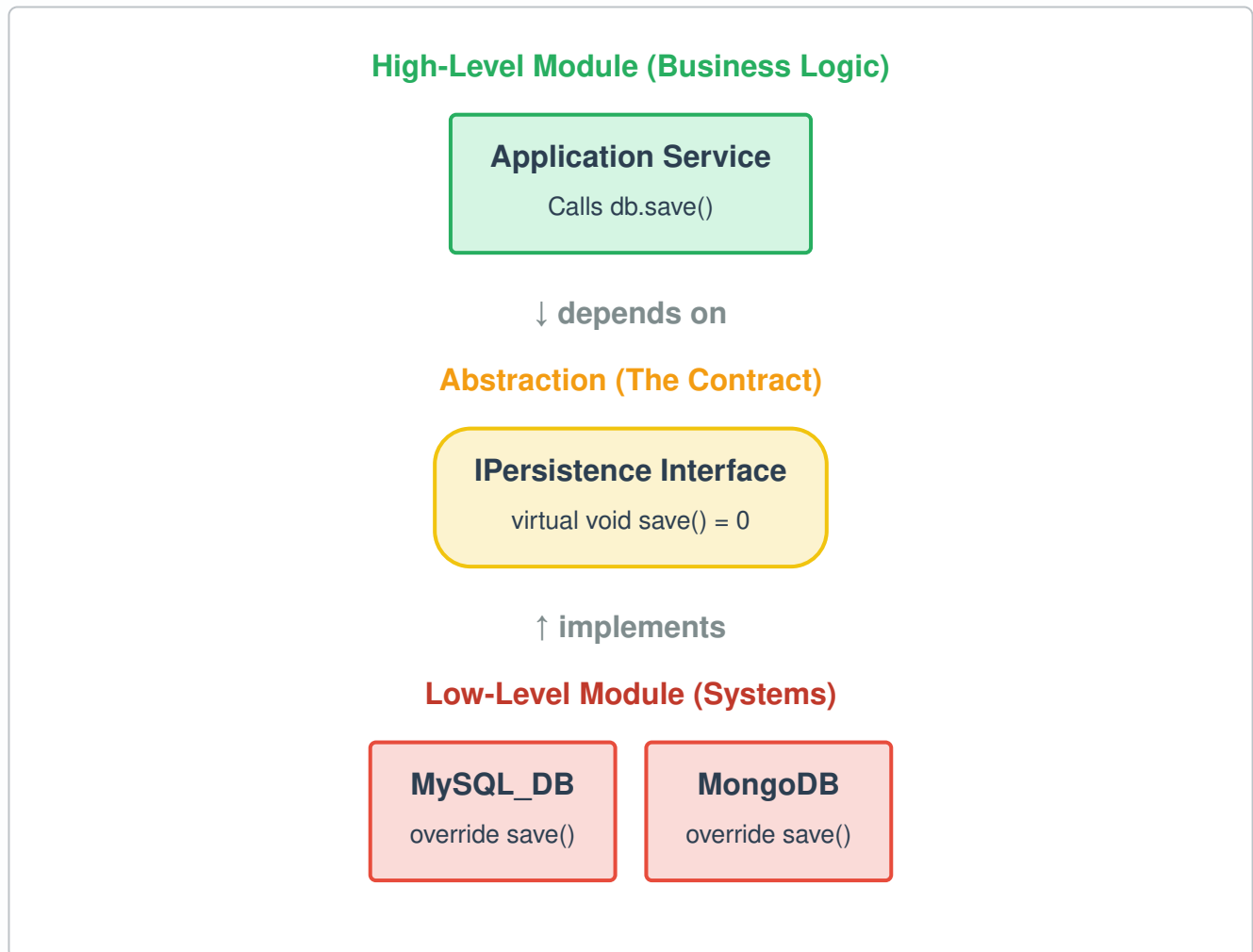


Pro-Tip: The Golden Rule of DIP

If the **Open/Closed Principle (OCP)** is your target destination, the **Dependency Inversion Principle (DIP)** is the vehicle that gets you there.

The Solution: Invert the Dependency via Abstraction

Instead of the Application depending directly on the Database, both the Application and the Database should depend on an Interface (a Contract).



Real-World Analogy: A CEO (High-level) does not walk to a Junior Developer's desk (Low-level) to assign a specific coding task. Instead, the CEO communicates the requirement to a Manager (Abstraction). The Manager then interfaces with whichever Developers are currently on the team. If the developers change, the CEO's workflow remains entirely uninterrupted.

4. Conclusion: Principles, Not Laws

It is vital to remember that SOLID represents **principles**, not rigid laws. In object-oriented programming, we map real-world entities into code. A real-world User might eat, sleep, and work, but an Uber app only cares about a user's ability to `bookRide()`, while a Swiggy app cares about `orderFood()`.

Sometimes, adhering 100% to SOLID will lead to over-engineering and unnecessary abstractions. Software architecture is about **trade-offs**. The goal is to follow these principles strictly enough to keep

your code clean, modular, and scalable, while being flexible enough to deliver business logic efficiently.